Roadmap: Build a Linux Process Visualizer in Rust (ratatui)

Target machine: Ubuntu in VMware. Everything in the setup phase is done from the terminal only.

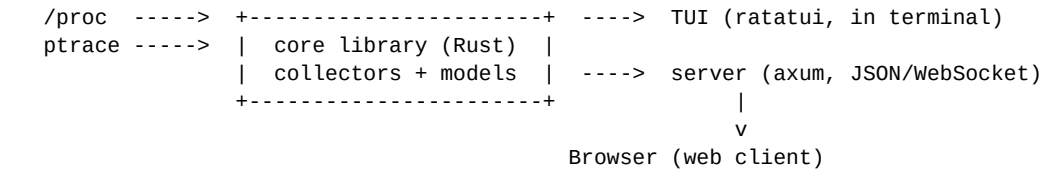
Working name for the project: **procscope**.

1. What you are building

A terminal app that shows what Linux processes are doing, going deeper than a normal monitor:

- A live process table (CPU, memory, threads, state), sortable and filterable.
- A drill-down view for the selected process: open files, network sockets, memory maps, I/O, threads.
- A live syscall view (like `strace`, with counters and timing).
- A web mode that shows the same data in a browser.

Architecture



Why a server in the middle: a web page runs in the browser sandbox and cannot read `/proc` or call `ptrace`. The Rust server reads the data on the VM and streams it to the page.

2. Important note about "web interface using ratatui"

`ratatui` draws to a terminal. It does not produce web pages by itself. You have two options:

- **Option A (closest to your idea):** compile a ratatui app to WebAssembly so it renders in the browser. As far as I know, a crate called **Ratzilla** is built for exactly this (ratatui in the browser via WASM). Check its current docs on GitHub and ratatui.rs before committing, since this area changes quickly.
- **Option B (most reliable):** a small `axum` server that serves JSON/WebSocket data plus a simple HTML/JavaScript page that draws tables and charts.

Recommended order: finish the TUI first, then build the server (needed by both options), then try Option A. If Option A fights you, fall back to Option B and you still have a working web mode.

3. Phase 0: Environment setup (terminal only)

Assume a fresh Ubuntu VM with nothing installed. Run these in order.

3.1 Check your system

```
uname -m          # x86_64 or aarch64. This matters later for syscall tracing.
lsb_release -a    # Ubuntu version
nproc && free -h  # CPU cores and RAM
df -h /           # free disk space (aim for 15 GB+ free)
```

Tip: Rust compiles slowly on tiny VMs. In VMware settings (with the VM powered off), give it 4 GB+ RAM and 2-4 CPU cores if your host allows it.

3.2 Update the system

```
sudo apt update && sudo apt upgrade -y
```

3.3 Install build tools and essentials

```
sudo apt install -y build-essential curl git pkg-config libssl-dev ca-certificates
```

`build-essential` provides the C compiler and linker that Rust needs. Without it you will see `linker 'cc' not found`.

3.4 VMware integration (clipboard, resizing, time sync)

```
sudo apt install -y open-vm-tools
```

If you use the Ubuntu desktop, also install `open-vm-tools-desktop`, then reboot the VM.

3.5 Configure Git

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

3.6 Install Rust with rustup

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Choose option 1 (default installation). Then load Cargo into your current shell:

```
. "$HOME/.cargo/env"
rustc --version
cargo --version
```

3.7 Add Rust developer tools

```
rustup component add rustfmt clippy rust-analyzer
```

- `rustfmt`: auto-formats code (`cargo fmt`)
- `clippy`: lints and teaches idiomatic Rust (`cargo clippy`)
- `rust-analyzer`: editor smarts, used by Neovim, Helix, VS Code and others

3.8 Pick a terminal editor

Easiest: use `nano`, which is already installed. A more powerful option:

```
sudo apt install -y neovim
```

3.9 Install Linux inspection and testing tools

```
sudo apt install -y strace htop tmux stress-ng man-db manpages manpages-dev
```

- `strace` and `htop` are the tools you are learning from and comparing against.
- `tmux` lets you split the terminal: editor in one pane, app in another, logs in a third.
- `stress-ng` creates fake load so you have something interesting to watch.
- The man pages give you the official documentation: `man 5 proc`, `man 2 ptrace`, `man 1 strace`.

3.10 Create the project and test it

```
mkdir -p ~/projects && cd ~/projects
cargo new procscope
cd procscope
cargo run
```

You should see `Hello, world!`. Then add the first dependencies:

```
cargo add ratatui anyhow
```

`ratatui` ships with a terminal backend (crossterm), so you do not need to add it separately. Because ratatui's API changes between versions, follow the tutorial at ratatui.rs for the version that `cargo add` installed.

3.11 Install later, when you reach the phase that needs it

```
# Phase 5 (syscall tracing)
cargo add nix --features ptrace,signal,process
# Phase 6 (web mode)
cargo add serde --features derive
cargo add serde_json
cargo add tokio --features full
cargo add axum --features ws
sudo apt install -y openssh-server          # to reach the VM from your host browser
rustup target add wasm32-unknown-unknown    # only for the WebAssembly option
cargo install trunk                         # only for the WebAssembly option (slow to build)
```

3.12 Setup checklist

- [ ] `cargo --version` works
- [ ] `cargo run` prints Hello, world!
- [ ] `strace ls` prints a stream of syscalls
- [ ] `man 5 proc` opens the /proc manual (press `q` to quit)
- [ ] `git status` works inside `~/projects/procscope`

4. Roadmap overview

Time estimates assume about 6-8 hours per week. Adjust freely.

Phase	Weeks	Goal	You are done when
0. Setup	0.5	Working Rust environment	Checklist above is complete
1. Rust basics	2	Read and write simple Rust	A CLI prints uptime, load and total RAM from /proc
2. /proc core	2	Library that reads process data	Unit tests pass on saved /proc samples
3. TUI v1	2	Live process table	You can sort, filter, select and quit cleanly
4. Drill-down	2	Per-process detail panels	Files, sockets, maps and I/O show for a selected PID
5. Syscall view	3	Live syscall trace	You can attach to a PID and see syscalls with counts
6. Web mode	3	Browser view	Your host browser shows live data from the VM
7. Polish	1-2	Shareable project	README, tests, clean clippy, demo recording

5. Phase 1: Rust basics (2 weeks)

Learn just enough Rust to start; the project will teach the rest.

- Read The Rust Programming Language (the official book, free at doc.rust-lang.org/book): chapters 1-10 first (ownership, structs, enums, error handling, generics, traits).
- Do the Rustlings exercises (`cargo install rustlings`, then `rustlings`).
- Concepts you must be comfortable with: ownership and borrowing, `Option` and `Result`, the `?` operator, `Vec`, `HashMap`, iterators, modules.

Mini project: a command-line program that reads and prints:

- uptime from `/proc/uptime`
- load average from `/proc/loadavg`
- `MemTotal` and `MemAvailable` from `/proc/meminfo`

Use only the standard library (`std::fs::read_to_string`). Done when it runs and handles a missing file without panicking.

6. Phase 2: /proc core library (2 weeks)

Put the logic in `src/lib.rs` (and modules) so both the TUI and the future server can reuse it.

What to read for each feature

Feature	Source in /proc
Process list	`/proc/<pid>/stat`, `/proc/<pid>/cmdline`, `/proc/<pid>/comm`
CPU %	`utime + stime` from `stat`, sampled twice, divided by clock ticks and elapsed time
Memory	`/proc/<pid>/status` (VmRSS, VmSize), `/proc/<pid>/statm`, `/proc/meminfo`
Threads	`/proc/<pid>/task/<tid>/`
Open files	`/proc/<pid>/fd/` (symlinks), `/proc/<pid>/fdinfo/`
Sockets	`/proc/net/tcp`, `tcp6`, `udp`, `unix`, matched by inode from `socket:[12345]` links
Memory maps	`/proc/<pid>/maps`, `/proc/<pid>/smaps_rollup`
I/O counters	`/proc/<pid>/io`
Namespaces, cgroups	`/proc/<pid>/ns/`, `/proc/<pid>/cgroup`
System totals	`/proc/stat`, `/proc/meminfo`, `/proc/loadavg`, `/proc/uptime`

Read `man 5 proc` for the exact field meanings.

Gotchas you will hit

- In `/proc/<pid>/stat`, the process name is in parentheses and can contain spaces or even parentheses. Split on the **last** `)`, not the first.
- Processes disappear between listing and reading. Treat "file not found" as normal, not an error.
- Other users' processes give "permission denied" for some files unless you run as root. Show a placeholder instead of crashing.
- CPU percent needs two samples. Store the previous reading and compute the difference.
- The clock tick rate (`CLK_TCK`) is almost always 100 on Linux; you can read it with `getconf CLK_TCK`.

Design suggestions

- Define clean data types: `ProcessInfo`, `FdInfo`, `SocketInfo`, `MemMap`, `SystemStats`.
- Return `Result<T, anyhow::Error>` from collectors and keep parsing functions pure (string in, struct out) so they are easy to test.
- Save real samples from your VM into `tests/fixtures/` and write unit tests against them (`cargo test`).
- First write all parsing by hand with `std::fs` to learn what the kernel exposes. Later, you may compare against the `procfs` crate to check your results.

7. Phase 3: TUI v1 (2 weeks)

Follow the ratatui tutorials at ratatui.rs, then build:

1. An event loop: draw, wait for a key or a timer tick (about 1 second), update, repeat.
2. A process table with columns PID, name, state, CPU %, memory, threads.

3. Keys: up/down to select, `s` to change sort column, `/` to filter by name, `q` to quit.
4. A header bar showing load average, memory use and uptime.

Critical details:

- Always restore the terminal on exit **and on panic**, otherwise your shell is left broken. (Recent ratatui versions provide helpers for this; check the docs for the version you installed.)
- Never block the UI thread on slow file reads. Collect data in a background thread and send it over a channel.
- The terminal owns your screen, so `println!` debugging will not work. Write logs to a file and watch them in another tmux pane: `tail -f procscope.log`.
- Test with load: `stress-ng --cpu 2 --timeout 60s` in another pane.

8. Phase 4: Drill-down panels (2 weeks)

When the user presses Enter on a process, open a detail screen with tabs:

- **Files:** list `/proc/<pid>/fd` symlink targets (files, pipes, sockets).
- **Network:** join socket inodes from the fd list with `/proc/net/tcp` and friends to show addresses, ports and connection state.
- **Memory:** parse `/proc/<pid>/maps` (heap, stack, libraries), with sizes and permissions.
- **I/O:** read and write bytes from `/proc/<pid>/io`, shown as rates.
- **Threads:** list `/proc/<pid>/task`, with per-thread state.

Add small visual touches with ratatui widgets: gauges for memory, sparklines for CPU history, colors for process state.

Test it on something that does real work, for example `curl` downloading a file, or a browser or database server if you have one.

9. Phase 5: Syscall view (3 weeks)

This is the hardest and most educational phase. Do it in two steps.

Step 1: wrap `strace` (week 1)

Run `strace -f -tt -p <pid>` as a child process, read its output line by line, parse each line into a struct (time, syscall name, arguments, return value, duration), and show it in a scrolling panel. Add a counter table, similar to `strace -c`. This gets you a working feature quickly.

Step 2: use `ptrace` yourself (weeks 2-3)

Replace the `strace` wrapper with your own tracer using the `nix` crate (and `libc` where needed):

1. Attach to a process (`PTRACE_SEIZE` or `PTRACE_ATTACH`).
2. Resume it with `PTRACE_SYSCALL` so it stops at each syscall entry and exit.
3. Read the CPU registers to get the syscall number and arguments.
4. Map syscall numbers to names with a lookup table (look for a syscall-table crate, or generate your own table).
5. Decode a few common arguments, such as file paths for `open`/`openat`, by reading the traced process's memory.

Know before you start:

- **Architecture matters.** Run `uname -m`. Register names and syscall numbers differ between x86_64 and aarch64 (ARM64). If your VM is ARM64, do not copy x86_64 tutorials literally.
- **Ubuntu restricts ptrace.** By default (`kernel.yama.ptrace_scope = 1`), you can only trace your own child processes. To attach to an arbitrary PID, run your tool with `sudo`. In a throwaway VM you can relax the setting temporarily: `sudo sysctl -w kernel.yama.ptrace_scope=0`.
- **Only one tracer per process.** If `strace` or `gdb` is already attached, your attach fails.
- Tracing slows the target process down. This is normal.
- Only trace processes on your own VM that you are allowed to inspect.

Useful reading: `man 2 ptrace`, `man 1 strace`, and Julia Evans' strace zine.

10. Phase 6: Web mode (3 weeks)

Step 1: split into a library and several programs

Your package can have a library plus multiple binaries:

```
procscope/
  Cargo.toml
  src/
    lib.rs          <- core: collectors and data types
    main.rs         <- the TUI
    bin/
      server.rs     <- the web server
  web/              <- browser client (separate crate or HTML, added later)
```

When you add a WebAssembly client you will probably need a Cargo workspace, because the client is compiled for a different target.

Step 2: a data server with axum

- `GET /api/processes` returns a JSON list.
- `GET /api/processes/<pid>` returns the drill-down data.
- A WebSocket endpoint pushes updates every second.
- Derive `serde::Serialize` on your core types so they convert to JSON easily.
- Bind to `127.0.0.1` by default. This data is sensitive (process names, files, connections) and there is no login. Do not expose it on `0.0.0.0` unless you add authentication.

Step 3: view it from your host browser

Use an SSH tunnel so the server stays private:

```
# on the VM: find its IP
ip -4 addr show
# on your host machine:
ssh -L 8080:localhost:8080 your-user@VM_IP
# then open http://localhost:8080 in your host browser
```

Step 4: the client

- **Option A:** a ratatui WebAssembly client (Ratzilla or similar) that connects to the WebSocket and renders the same widgets as the TUI.
- **Option B:** a plain HTML page with a little JavaScript for the table and a chart.

Start with Option B for a day if you want a quick win, then attempt Option A.

11. Phase 7: Polish and share (1-2 weeks)

- Run `cargo fmt` and `cargo clippy` and fix the warnings.
- Write tests for the parsers and the CPU calculation.
- Write a README with a screenshot or terminal recording (`sudo apt install asciinema`).
- Build an optimized binary: `cargo build --release`, then try `cargo install --path .`.
- Add a `--help` message and flags such as `--interval`, `--pid`, and `--port` (the `clap` crate is the standard choice).
- Push to GitHub. This is good portfolio material, since it shows systems knowledge and Rust.

12. Common problems and fixes

Problem	Likely cause and fix
`linker 'cc' not found`	`sudo apt install build-essential`
`cargo: command not found`	Run `. "$HOME/.cargo/env"` or open a new terminal
OpenSSL / pkg-config errors	`sudo apt install pkg-config libssl-dev`
Terminal broken after a crash	Type `reset` and press Enter; fix panic handling in your app
Permission denied reading /proc	Normal for other users' processes; handle it or use `sudo`
`ptrace: Operation not permitted`	Yama ptrace_scope or another tracer is attached; see Phase 5
Very slow builds	Give the VM more cores/RAM; use `cargo check` while editing
Monitor uses too much CPU itself	Refresh once per second and avoid re-reading unchanged data

13. Stretch goals

- Container view: group processes by cgroup and namespace.
- Process tree view using the PPID relationships.
- Network bandwidth per process.
- Alerts when a process crosses a CPU or memory threshold.
- Export a snapshot to JSON or a flame-style summary of syscalls.
- Advanced: explore eBPF with the `aya` crate for low-overhead tracing.

14. Learning resources

- The Rust Programming Language (doc.rust-lang.org/book)
- Rustlings exercises
- ratatui.rs (tutorials, examples, widget docs)
- docs.rs for each crate you use
- Man pages: `man 5 proc`, `man 2 ptrace`, `man 1 strace`
- Julia Evans' zines on strace and Linux debugging tools